```
    . $components['query'] . '#'
    . $components['fragment']
;
```

```
Output:
https://www.cats.com:8080/nutters/jess?status=hungry#meow
```

Be aware that `parse_url()` does *not* validate the string as a URL, and passing a malformed URL into the function can result in complete nonsense being returned! We will look at data validation later in the chapter.

Things get a little more complicated when manipulating query data. You might be tempted to simply have at it with regular expressions or other string manipulation functions, but I don't recommend it. I've fixed numerous bugs over the years where edge cases in such code have broken URL creation in PHP code. Far better to work with structured data and generate your own, and for this we have the built-in `parse_str()` and `http_build_query()`:

```
$url = 'https://localhost/whatever?complex[]=data&complex[]=here';
parse_str(parse_url($url, PHP_URL_QUERY), $query);
var_dump($query);
```

```
Output:
array(1) {
  ["complex"]=>
  array(2) {
    [0]=>
    string(4) "data"
    [1]=>
    string(4) "here"
  }
}
```

Notice the `$query` variable: this is passed by reference, and if present, it will receive the associative array of parsed data; otherwise, PHP will create a set of variables (with names matching those in the query data) in the current scope, which is generally not a great idea. Use `http_build_query()` to assemble a string from structured data: